



SEMANA 10

ADMINISTRACIÓN ESENCIAL



ALUMNO

Kevin Alfredo Macha Bruno



DOCENTE

Fernández Bejarano Raul



ANÁLISIS

Estudiamos el entorno
y los factores clave para
tomar decisiones
acertadas.



DISEÑO

Planteamos estrategias
creativas y efectivas
para lograr ventaja
competitiva.



GESTIÓN

Organizamos recursos
y tiempos para
optimizar cada
proceso.



SOLUCIÓN

Implementamos acciones
precisas para alcanzar
los objetivos
propuestos.



KEVIN ALFREDO MACHA BRUNO

PROPIEDADES Y CONFIGURACIONES DE BASES DE DATOS EN MICROSOFT SQL SERVER

SQL Server permite administrar y configurar las bases de datos para garantizar rendimiento, seguridad, disponibilidad y un crecimiento controlado.

1.1 OBJETIVOS DE APRENDIZAJE



Comprender la arquitectura de instancias y componentes de servicio del motor de Microsoft SQL Server.



Identificar la función analítica de los archivos físicos primarios (.mdf), secundarios (.ndf) y de registro de transacciones (.ldf).



Configurar parámetros de almacenamiento óptimos como tamaño inicial, límites máximos de crecimiento y tasas de incremento (PERCENTGROWTH).



Alterar las colaciones de texto (Collation) y los niveles de compatibilidad de la base de datos controlando el aislamiento de usuarios concurrentes.

1.2 ARQUITECTURA GENERAL DE SQL SERVER

Microsoft SQL Server opera bajo un modelo basado en instancias de servicio. Una instancia es una instalación completa del motor de base de datos que posee sus propios archivos ejecutables, registros de configuración de memoria, asignación de procesadores y bases de datos del sistema (master, model, msdb, tempdb).

En el corazón del sistema se encuentra el Database Engine (Motor de Base de Datos), encargado del almacenamiento físico, procesamiento transaccional, seguridad de acceso y ejecución de las directivas declaradas en T-SQL.

COMPONENTES PRINCIPALES



Database Engine
(Motor de Base de Datos)
Procesamiento, almacenamiento y transacciones.



Seguridad
Control de acceso y autenticación.



Procesadores
Ejecución de consultas y procesos.



Memoria
Configuraciones y asignación de recursos.



Bases del Sistema
master, model, msdb, tempdb.

Servicios complementarios:

SQL Server Agent (Tareas programadas) | SQL Server Browser | Protocolos de instancia

1.3 PROPIEDADES FUNDAMENTALES DEL ALMACENAMIENTO FÍSICO

Una base de datos en SQL Server no es una sola entidad monolítica en disco; está compuesta al menos por dos tipos de archivos físicos distribuidos:



Archivo de Datos Primario (.mdf)

Contiene las tablas del sistema, los metadatos, programas, vistas, archivos del sistema y el historial operativo. Cada base de datos posee exactamente un archivo .mdf.



Archivos de Datos Secundarios (.ndf)

Son opcionales y permiten extender el almacenamiento de la información de las tablas en múltiples datos físicos. Se asocian a grupos de archivos previamente definidos.



Archivo de Registro de Transacciones (.ldf)

Almacena todas las actividades realizadas con las transacciones: lecturas y escritura de índices directamente, registra el estado de las datos "antes" y "después" de cada transacción.

MAPA DE ARQUITECTURA



1.4 GUÍA DE CREACIÓN ESTRUCTURADA MEDIANTE T-SQL

```

USE master;
GO
CREATE DATABASE [NOMBRE_BD]
ON
PRIMARY (
  NAME = N'[NOMBRE_BD]_Data',
  FILENAME = N'C:\Datos\[NOMBRE_BD]_Data.mdf',
  SIZE = 10MB,
  MAXSIZE = UNLIMITED,
  FILEGROWTH = 10%
),
FILEGROUP [FG_Datos] (
  NAME = N'[NOMBRE_BD]_Secundario',
  FILENAME = N'C:\Datos\[NOMBRE_BD]_Sec.ndf',
  SIZE = 10MB,
  MAXSIZE = UNLIMITED,
  FILEGROWTH = 10%
)
LOG ON (
  NAME = N'[NOMBRE_BD]_Log',
  FILENAME = N'C:\Logs\[NOMBRE_BD]_Log.ldf',
  SIZE = 10MB,
  MAXSIZE = 2048MB,
  FILEGROWTH = 10%
);
GO
  
```

ASPECTOS A CONSIDERAR



Rutas adecuadas y con respaldo.



Nombres claros y consistentes.



Espacio suficiente y control del crecimiento.



Verificar permisos de escritura del servicio SQL Server.

1.5 MODIFICACIÓN DE PROPIEDADES Y BUENAS PRÁCTICAS

Modificar las propiedades de una base de datos en SQL Server es una necesidad para mitigar problemas de rendimiento y garantizar la estabilidad del servicio de datos.

BUENAS PRÁCTICAS



Separación de Capas de I/O:

En ambientes críticos, colocar los archivos de datos (.mdf, .ndf) en discos físicos diferentes de los archivos de registro de transacciones (.ldf).



Evitar el Crecimiento por Porcentajes:

Nunca use FILEGROWTH por porcentaje en bases de datos grandes; esto puede causar ineficiencias significativas en forma de fragmentación y bloqueos.



Monitoreo Constante: Usar DMVs, informes de rendimiento y alertas del Agente SQL para anticipar el desborde de espacio o problemas de contención de recursos.



Backups Regulares: Asegurar copias de seguridad completa, diferencial y del log de transacciones.

1.6 CONSIDERACIONES ADICIONALES



Collation:

Define las reglas de comparación y ordenación del texto. Cambiarla puede afectar búsquedas, índices y compatibilidad.



Nivel de compatibilidad:

Controla características disponibles del motor SQL Server para garantizar la correcta ejecución de consultas anteriores.



Aislamiento de transacciones:

Configuración del nivel de aislamiento (READ COMMITTED, SNAPSHOT, SERIALIZABLE) para bloquear, mostrar errores y proteger la consistencia de los datos.

Una correcta configuración de las bases de datos es clave para asegurar rendimiento, escalabilidad y disponibilidad en entornos empresariales exigentes.

CAPÍTULO 2

TIPOS DE RECUPERACIÓN (SIMPLE, FULL, BULK-LOGGED)

Los modelos de recuperación de SQL Server determinan cómo se administran las transacciones y los registros de transacciones (Log), afectando el mantenimiento, el rendimiento y la recuperación ante fallos.

2.1 OBJETIVOS DE APRENDIZAJE



Diferenciar las implicaciones transaccionales y de mantenimiento de los tres modelos de recuperación soportados en SQL Server.



Administrar el comportamiento físico y lógico del archivo de registro de transacciones (.ldf).



Diseñar e implementar scripts avanzados de respaldo completo, diferencial y transaccional (BACKUP DATABASE / LOG).



Ejecutar planes de restauración estructurados ante configuraciones o corrupción de datos lógicos.

2.2 ANÁLISIS DE LOS TRES MODELOS DE RECUPERACIÓN

El Modelo de Recuperación (Recovery Model) es una propiedad de configuración global de la base de datos que determina la forma en que SQL Server gestiona el almacenamiento del archivo de registro de transacciones (.ldf).

CARACTERÍSTICA TÉCNICA	SIMPLE	FULL	BULK-LOGGED
Transacciones en el Log	Si, hasta cada Checkpoint de datos.	Si, registra cada transacción de log formal.	Si, registra con excepción de log formal.
Riesgo de Crecimiento	Muy Bajo.	Muy Alto (si no se realiza respaldo).	Moderado.
Respaldos de Log	No soportados.	Si, indispensables.	Si, soportados.
Restauración Punto en Tiempo	No (solo hasta el último backup completo).	Si (Permite un registro exacto).	Si, excepto transacciones masivas.



2.3 SCRIPTS AVANZADOS DE RESPALDO Y RESTAURACIÓN DE EMERGENCIA



1. Asegurar el modelo FULL

```
ALTER DATABASE GradosTitulos  
SET RECOVERY FULL;
```



2. Respaldo Completo Base

```
BACKUP DATABASE GradosTitulos  
TO DISK = 'D:\Backups\Full_GradosTitulos.bak'  
WITH FORMAT, INIT, NAME = 'Full Backup -  
GradosTitulos', STATS = 10;
```



3. Respaldo Diferencial

```
BACKUP DATABASE GradosTitulos  
TO DISK = 'D:\Backups\Diff_GradosTitulos.bak'  
WITH DIFFERENTIAL, INIT, NAME = 'Diff Backup -  
GradosTitulos', STATS = 10;
```



4. Respaldo de Log de Transacciones

```
BACKUP LOG GradosTitulos  
TO DISK = 'D:\Backups\Log_GradosTitulos.trn'  
WITH INIT, NAME = 'Log Backup - GradosTitulos',  
STATS = 10;
```



2.4 ADMINISTRACIÓN DEL COMPORTAMIENTO DEL ARCHIVO .LDF

El archivo de registro de transacciones (.ldf) registra todas las operaciones que modifican los datos. Su comportamiento varía según el modelo de recuperación:



Crece continuamente en modelos FULL y BULK-LOGGED hasta realizar backups del log.



Se trunca en modelo SIMPLE automáticamente en cada checkpoint.



Requiere mantenimiento periódico para evitar que ocupe todo el disco y asegurar un rendimiento óptimo.



Monitorear el log adecuadamente asegura un rendimiento y disponibilidad de recuperación.

2.5 Secuencia de Restauración ante una Pérdida Lógica de Datos

1

Base de datos

```
USE master;  
GO  
ALTER DATABASE GradosTitulos  
SET SINGLE_USER  
WITH ROLLBACK  
IMMEDIATE;  
ALTER DATABASE GradosTitulos  
DROP;  
GO
```

2

Diferencial (si existe)

```
RESTORE DATABASE  
GradosTitulos  
FROM DISK =  
'D:\Backups\  
Diff_GradosTitulos.bak'  
WITH RECOVERY,  
REPLACE;  
GO
```

3

Logs (en orden)

```
RESTORE LOG  
GradosTitulos  
FROM DISK =  
'D:\Backups\  
Log_*.trn'  
WITH RECOVERY;  
GO
```

-- Repetir hasta el último log disponible

4

Restaurar y volver a línea

```
ALTER DATABASE  
GradosTitulos  
SET MULTI_USER;  
GO
```



Base de datos recuperada y operativa



Elegir el modelo correcto depende del balance entre rendimiento, espacio en disco y nivel de recuperación requerido.

Para entornos críticos que requieren Punto en el Tiempo, use FULL.

Para alta carga con mínima recuperación, use SIMPLE.

Kevin Alfredo Macha Bruno



ADMINISTRACIÓN DE USUARIOS Y ROLES DE SEGURIDAD

La seguridad en SQL Server protege la información mediante la autenticación de usuarios y la autorización de accesos a objetos y datos.

3.1 OBJETIVOS DE APRENDIZAJE

- Comprender la arquitectura jerárquica de los niveles del modelo de seguridad de SQL Server (Login vs. User).
- Configurar cuentas de inicio de sesión utilizando los modos de Autenticación de Windows y Autenticación de SQL Server.
- Crear y asociar usuarios de base de datos vinculándolos a los esquemas de seguridad institucionales.
- Administrar privilegios de acceso mediante la asignación de roles fijos de servidor y roles fijos de base de datos.

3.2 EL MODELO DE SEGURIDAD: LOGINS VS. USUARIOS

La seguridad en SQL Server está estructurada en dos niveles independientes y complementarios: el nivel de acceso a la instancia (servidor) y el nivel de acceso a los contenedores lógicos de datos (bases de datos).

INICIO DE SESIÓN (LOGIN)

Es la entidad de seguridad configurada a nivel de la instancia del servidor. Su función es la Autenticación.

Se almacena en Base de datos master.

USUARIO DE BASE DE DATOS (USER)

Es la entidad de seguridad configurada dentro de una base de datos específica. Su función es la Autorización.

Se almacena en Cada base de datos de usuario.

RELACIÓN:

Un Login puede tener uno o varios Users en diferentes bases de datos.
Un User siempre está asociado a un Login.

3.3 IMPLEMENTACIÓN DE SEGURIDAD POR SCRIPTS T-SQL

1. CREACIÓN DE LOGINS

```
CREATE LOGIN [OperadorTransactolog]
WITH PASSWORD = '*****',
CHECK_EXPIRATION = OFF,
CREATE_DEFAULT_DATABASE = master;
GO
```

2. CREACIÓN Y MAPEO DE USUARIOS INTERNOS

```
USE GradosTítulos;
GO
-- Crear el usuario en la base de datos
CREATE USER [OperadorTransito]
FOR LOGIN [OperadorTransactolog];
GO
```

3. ASIGNACIÓN A ROLES FIJOS DE BASE DE DATOS

```
ALTER ROLE db_datareader
ADD MEMBER [OperadorTransito];

ALTER ROLE db_datawriter
ADD MEMBER [OperadorTransito];
```

4. ASIGNACIÓN A ROLES FIJOS DE SERVIDOR

```
ALTER ROLE sysadmin
ADD MEMBER [OperadorTransactolog];

ALTER ROLE securityadmin
ADD MEMBER [OperadorTransactolog];
```

BENEFICIOS DE UNA BUENA ADMINISTRACIÓN DE SEGURIDAD

- Protege la confidencialidad, integridad y disponibilidad de los datos.
- Controla el acceso según funciones y responsabilidades institucionales.
- Facilita la auditoría y el seguimiento de actividades de usuarios.
- Reduce riesgos de acceso no autorizado y mal uso de los datos.
- Mejora el cumplimiento de normativas y políticas de seguridad.

NIVELES DEL MODELO DE SEGURIDAD



ROLES FIJOS DE SERVIDOR (Ejemplos)

- sysadmin:** Control total del servidor.
- securityadmin:** Administra logins y roles.
- serveradmin:** Configuración del servidor.
- dbcreator:** Puede crear bases de datos.
- diskadmin:** Administra recursos de disco.

ROLES FIJOS DE BASE DE DATOS (Ejemplos)

- db_owner:** Control total de la base de datos.
- db_datareader:** Puede leer todos los datos.
- db_datawriter:** Puede insertar y actualizar datos.
- db_ddladmin:** Administra objetos (DDL).
- db_denydatareader:** Niega lectura de datos.

Elegir el modelo correcto depende del balance entre rendimiento, espacio en disco y nivel de recuperación requerido.

Para entornos críticos que requieren Punto en el Tiempo, use **FULL**.
Para alta carga con mínima recuperación, use **SIMPLE**.

ASIGNACIÓN DE PERMISOS Y POLÍTICAS DE ACCESO

SQL Server utiliza un sistema de permisos granulares y roles para controlar el acceso a objetos y datos, asegurando la seguridad y el cumplimiento de políticas organizacionales.

4.1 OBJETIVOS DE APRENDIZAJE

- Comprender la jerarquía del sistema de permisos granulares sobre esquemas y objetos en SQL Server.
- Dominar los comandos de control de acceso: GRANT, DENY y REVOKE.
- Configurar políticas de control de acceso basadas en roles de usuario (RBAC).
- Aplicar el principio de mínimo privilegio en el diseño de arquitecturas de bases de datos empresariales.

4.2 COMANDOS DE CONTROL DE ACCESO: GRANT, DENY Y REVOKE

Para gestionar la matriz de seguridad de la base de datos de manera granular, se utilizan tres comandos vel Transact-SQL.

GRANT (Conceder)	Otorga un permiso específico sobre un objeto o esquema a un usuario o rol.	Permite el acceso
DENY (Negar)	Bloquea explícitamente un acceso a un usuario, incluso si tiene permisos por roles.	Niega el acceso
REVOKE (Revocar)	Elimina un permiso que ha sido otorgado previamente, dependiendo del contexto.	Quita el permiso

Prioridad de permisos: DENY > GRANT
Si un permiso es concedido (GRANT) y luego denegado (DENY), la denegación prevalece.

4.3 CONFIGURACIÓN DE POLÍTICAS RBAC PARA EL CONTROL ORGANIZACIONAL

El modelo RBAC (Role-Based Access Control) simplifica la administración de permisos asignando roles a los usuarios. Los roles agrupan permisos según funciones o responsabilidades.



- | | | | |
|--|--|---|--|
| 1. Crear Roles | 2. Asignar Usuarios | 3. Asignar Permisos | 4. Auditoría y Revisión |
| Define roles que representen funciones del negocio (Ejemplo: Analista, Operador, Administrador). | Asocia usuarios o grupos de Windows/AD a los roles correspondientes. | Concede permisos a los roles sobre esquemas o bases de datos. | Monitorea y revisa permisos periódicamente para garantizar el cumplimiento de políticas. |

Principio de Mínimo Privilegio
Otorgar solo los permisos estrictamente necesarios para que los usuarios puedan realizar sus funciones. Reduce riesgos de seguridad y errores operativos.

JERARQUÍA DE PERMISOS EN SQL SERVER

Los permisos se heredan y pueden aplicarse en distintos niveles de la jerarquía.

- Servidor**
Permisos a nivel de instancia
Ej: CONNECT SQL
- Base de Datos**
Permisos a nivel de base de datos
Ej: CREATE TABLE
- Esquema**
Permisos a nivel de esquema
Ej: SELECT, INSERT
- Objeto**
Permisos a nivel de objeto
Ej: EXECUTE, UPDATE

- Tipos de permisos comunes:**
- | | |
|---|---|
| ☒ SELECT: Leer datos | ☒ SELECT: Eliminar datos |
| ☒ INSERT: Insertar datos | ☒ EXECUTE: Ejecutar objetos |
| ☒ UPDATE: Modificar datos | ☒ REFERENCES: Referenciar objetos |
| ☒ DELETE: Eliminar datos | ☒ CONTROL: Control total del objeto |

EJEMPLOS DE COMANDOS DE CONTROL DE ACCESO

- ```
-- 1. Conceder permisos
GRANT SELECT, INSERT ON Esquema.Tabla TO AnalistaRol;

-- 2. Denegar permisos
DENY DELETE ON Esquema.Tabla TO AnalistaRol;

-- 3. Revocar permisos
REVOKE INSERT ON Esquema.Tabla FROM AnalistaRol;

-- 4. Transferir permisos entre roles
GRANT EXECUTE ON Procedimiento TO OperadorRol;

-- 5. Control total de un objeto
GRANT CONTROL ON Esquema.Vista TO AdminRol;
```

## EJEMPLO DE POLÍTICA RBAC EN UNA ORGANIZACIÓN

| Rol           | Descripción                            | Permisos Asignados                  | Usuarios Ejemplo     |
|---------------|----------------------------------------|-------------------------------------|----------------------|
| Analista      | Puede leer e insertar datos de ventas. | SELECT, INSERT en tablas de ventas  | analista1, analista2 |
| Operador      | Puede actualizar datos operativos.     | SELECT, UPDATE en tablas operativas | operador1, operador2 |
| Supervisor    | Controla y revisa la base de datos.    | CONTROL en bases de datos asignadas | dba_sp, dba_bd       |
| Administrador | Acceso total para gestión y seguridad. | SELECT en todas las tablas          | admin1, admin2       |

Una correcta asignación de permisos y políticas RBAC asegura la confidencialidad, integridad y disponibilidad de los datos, cumpliendo con los estándares de seguridad y políticas organizacionales.

Elegir el modelo correcto depende del balance entre rendimiento, espacio en disco y nivel de recuperación requerido.

Para entornos críticos que requieren Punto en el Tiempo, use **FULL**.

Para alta carga con mínima recuperación, use **SIMPLE**.



# MONITOREO BÁSICO CON EL SQL SERVER ACTIVITY MONITOR

El **Activity Monitor** de SQL Server permite supervisar en tiempo real el rendimiento y la actividad del servidor, ayudando a identificar cuellos de botella y optimizar el uso de recursos.

## 5.1 OBJETIVOS DE APRENDIZAJE



Acceder e interpretar las métricas de rendimiento en tiempo real a través del Activity Monitor de SSMS.



Identificar y solucionar cuellos de botella provocados por un uso excesivo del procesador (CPU) o bloqueos transaccionales entre conexiones.



Analizar los tiempos de espera y el impacto de las operaciones de lectura y escritura en los discos del servidor.



Identificar y analizar consultas con un alto costo de ejecución (Recent Expensive Queries) para aplicar optimizaciones avanzadas.

## 5.2 COMPONENTES DE ANÁLISIS DEL ACTIVITY MONITOR

El Monitor de Actividad proporciona información gráfica y en tiempo real sobre el comportamiento operativo interno de la instancia de SQL Server a través de sus paneles principales:



### OVERVIEW:

Gráficas dinámicas de tiempo de procesador (% Processor Time), solicitudes en espera (Waiting Requests) y operaciones de entrada/salida (Database I/O).



### PROCESSES:

Listado detallado de todas las operaciones activas de los usuarios en el servidor (Session ID, SPID). Muestra la columna crítica "Blocked By".

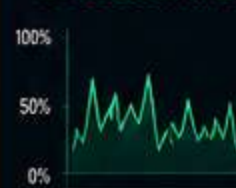


### RECENT EXPENSIVE QUERIES:

Sentencias que han consumido la mayor cantidad de recursos de cómputo en los últimos minutos del servicio.

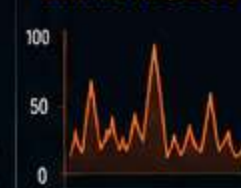
## 5.2.1 OVERVIEW: GRÁFICAS EN TIEMPO REAL

### % PROCESSOR TIME



Muestra el porcentaje de uso del CPU del servidor.

### WAITING REQUESTS



Número de solicitudes en espera. Indica posibles bloqueos o cuellos de botella.

### DATABASE I/O



Operaciones de entrada/salida en los discos del servidor.



Estas gráficas se actualizan en tiempo real y permiten detectar rápidamente picos de carga, esperas y problemas de I/O.

## 5.2.2 PROCESSES: CONEXIONES ACTIVAS

Nos muestra información detallada de cada sesión conectada a la instancia de SQL Server. La columna "Blocked By" es clave para identificar bloqueos.

| SESSION ID | LOGIN    | DATABASE     | STATUS    | CPU (%) | MEMORY (KB) | COMMAND | BLOCKED BY |
|------------|----------|--------------|-----------|---------|-------------|---------|------------|
| 51         | usuario1 | VentasDB     | running   | 12.3    | 5,170       | SELECT  | 0          |
| 52         | usuario2 | VentasDB     | suspended | 8.8     | 2,048       | UPDATE  | 51         |
| 53         | usuario3 | ReportesDB   | running   | 5.6     | 1,840       | INSERT  | 0          |
| 54         | usuario4 | InventarioDB | running   | 2.1     | 1,359       | SELECT  | 0          |
| 55         | usuario5 | VentasDB     | suspended | 0.0     | 1,034       | SELECT  | 52         |



Si "Blocked By" es diferente de 0, la sesión está bloqueada por otra conexión. Identifica el Session ID que está causando el bloqueo para investigarlo.

## 5.2.3 RECENT EXPENSIVE QUERIES

Muestra las consultas que más recursos de cómputo han consumido durante los últimos minutos.

| QUERY                  | CPU (%) | READS   | WRITES | DURATION (MS) | EXECUTIONS COUNT |
|------------------------|---------|---------|--------|---------------|------------------|
| SELECT * FROM Ventas   | 78.0    | 251,345 | 219    | 8,200         | 12               |
| UPDATE Inventario SET  | 75.0    | 145,211 | 278    | 6,300         | 8                |
| INSERT INTO Reportes   | 55.0    | 123,915 | 986    | 4,800         | 6                |
| SELECT * FROM Clientes | 42.0    | 98,114  | 145    | 3,420         | 9                |
| DELETE FROM Logs WHERE | 30.0    | 74,387  | 102    | 2,450         | 4                |



Úsalo para identificar las consultas más costosas y mejorar planes de ejecución o índices.

## 5.3 IDENTIFICACIÓN DE BLOQUEOS MEDIANTE T-SQL

Puedes consultar las sesiones bloqueadas y bloqueantes utilizando T-SQL.

```
SELECT
 r.session_id AS Bloqueada,
 r.wait_type,
 r.blocking_session_id AS Bloqueante,
 s.login_name,
 s.host_name,
 r.status
FROM sys.dm_exec_requests r
JOIN sys.dm_exec_sessions s
ON r.session_id = s.session_id
WHERE r.blocking_session_id <> 0;
```



Esta consulta ayuda a identificar qué sesión está bloqueando a otra y qué recurso está causando el bloqueo.

## JERARQUÍA DE PERMISOS EN SQL SERVER

Los permisos se heredan y pueden aplicarse en distintos niveles de la jerarquía.



### Servidor

Permisos a nivel de instancia  
Ej: CONNECT SQL



### Base de Datos

Permisos a nivel de base de datos  
Ej: CREATE TABLE



### Esquema

Permisos a nivel de esquema  
Ej: SELECT, INSERT



### Objeto

Permisos a nivel de objeto  
Ej: EXECUTE, UPDATE

## EJEMPLOS DE COMANDOS DE CONTROL DE ACCESO



### 1. Conceder permisos

```
GRANT SELECT, INSERT ON Esquema.Tabla
TO AnalistaRol;
```



### 2. Denegar permisos

```
DENY DELETE ON Esquema.Tabla
TO AnalistaRol;
```



### 3. Revocar permisos

```
REVOKE INSERT ON Esquema.Tabla
FROM AnalistaRol;
```



### 4. Transferir permisos entre roles

```
GRANT EXECUTE ON Procedimiento TO OperadorRol;
```



### 5. Control total de un objeto

```
GRANT CONTROL ON Esquema.Vista TO AdminRol;
```

## EJEMPLO DE POLÍTICA RBAC EN UNA ORGANIZACIÓN

| ROL                                                                                                 | DESCRIPCIÓN                            | PERMISOS ASIGNADOS                  | USUARIOS EJEMPLO     |
|-----------------------------------------------------------------------------------------------------|----------------------------------------|-------------------------------------|----------------------|
|  Analista      | Puede leer e insertar datos de ventas. | SELECT, INSERT en tablas de ventas  | analista1, analista2 |
|  Operador      | Puede actualizar datos operativos.     | SELECT, UPDATE en tablas operativas | operador1, operador2 |
|  Supervisor    | Controla y revisa la base de datos.    | CONTROL en bases de datos asignadas | dba_sp, dba_bd       |
|  Administrador | Acceso total para gestión y seguridad. | SELECT en todas las tablas          | admin1, admin2       |



Monitorea regularmente el Activity Monitor para mantener el rendimiento óptimo del servidor. Detectar problemas a tiempo evita impactos en los usuarios y en las aplicaciones.





## CAPÍTULO 6

# INTRODUCCIÓN AL USO DE SQL SERVER AGENT

### (TAREAS AUTOMÁTICAS)



## ¿QUÉ ES SQL SERVER AGENT?

Es un servicio de SQL Server que ejecuta y administra tareas programadas (trabajos) como copias de seguridad, limpieza de datos, ejecución de scripts, envío de alertas, entre otras.

- ◆ Ejecuta trabajos en horarios definidos.
- ◆ Permite crear procesos recurrentes de mantenimiento.
- ◆ Mejora el rendimiento y la disponibilidad de las bases de datos.
- ◆ Se administra desde SQL Server Management Studio (SSMS).



## 6.1 OBJETIVOS DE APRENDIZAJE

- Comprender la arquitectura operativa de automatización de servicios mediante SQL Server Agent.
- Diseñar y programar trabajos (jobs) estructurados compuestos por múltiples pasos teóricos (steps).
- Configurar programaciones horarias automáticas completas para tareas recurrentes de mantenimiento.
- Implementar alertas automáticas y notificar operadores para la gestión de incidentes en el servidor.

## 6.2 AUTOMATIZACIÓN DE MANTENIMIENTO MEDIANTE JOBS DEL AGENT

A continuación, se detalla un script transaccional para la automatización de la generación de respaldo diario con codificación de fecha ISO y la purga semanal de registros obsoletos en el sistema de gestión de la universidad.

```
-- PASO 1: SCRIPT DE RESPALDO DIARIO DINÁMICO (PASO 1 DEL JOB)
DECLARE @FechaEstilo varchar(20);
SET @FechaEstilo = CONVERT(varchar(20), GETDATE(), 112) + ".bak";

BACKUP DATABASE GradosTitulos
TO DISK = D:\Backups\GradosTitulos_Auto_" + @FechaEstilo
WITH INIT, COMPRESSION, STATS = 10;
GO

-- PASO 2: SCRIPT DE PURGA SEMANAL DE REGISTROS (PASO 2 DEL JOB)
DELETE FROM Transito.OMovimientos
WHERE Fecha < DATEADD(DAY, -7, CONVERT(DATE, GETDATE()));
GO
```

## ¿CÓMO AUDITAR Y VERIFICAR LOS JOBS?

- 1 En SSMS, expande la carpeta "SQL Server Agent".
- 2 Haz clic derecho sobre el trabajo que deseas auditar.
- 3 Selecciona la opción "View History".

En esta ventana podrás consultar todas las ejecuciones del job, incluyendo fecha, hora, estado (éxito o falla) y mensajes detallados.

| Date/Time           | Step ID | Step Name       | Duration | Sql Severity | Message                              |
|---------------------|---------|-----------------|----------|--------------|--------------------------------------|
| 2024-04-20 02:00:10 | 1       | Respaldo Diario | 00:00:27 | 0            | El paso se completó correctamente.   |
| 2024-04-20 02:00:45 | 2       | Purga Semanal   | 00:00:15 | 0            | El paso se completó correctamente.   |
| 2024-04-19 02:00:10 | 1       | Respaldo Diario | 00:00:25 | 0            | El paso se completó correctamente.   |
| 2024-04-19 02:00:40 | 2       | Purga Semanal   | 00:00:16 | 0            | El paso se completó correctamente.   |
| 2024-04-19 02:00:10 | 1       | Respaldo Diario | 00:00:32 | 16           | Error al escribir en el dispositivo. |

## COMPONENTES CLAVE DE SQL SERVER AGENT



## BENEFICIOS DEL USO DE SQL SERVER AGENT

- Reduce el trabajo manual y el riesgo de errores.
- Garantiza la ejecución oportuna de tareas críticas.
- Optimiza el rendimiento y la disponibilidad del servidor.
- Permite una respuesta rápida ante incidentes.



## JERARQUÍA DE PERMISOS EN SQL SERVER

Los permisos se heredan y pueden aplicarse en distintos niveles de la jerarquía.

- Servidor**  
Permisos a nivel de instancia  
Ej: CONNECT SQL
- Base de Datos**  
Permisos a nivel de base de datos  
Ej: CREATE TABLE
- Esquema**  
Permisos a nivel de esquema  
Ej: SELECT, INSERT
- Objeto**  
Permisos a nivel de objeto  
Ej: EXECUTE, UPDATE

## EJEMPLOS DE COMANDOS DE CONTROL DE ACCESO

- 1. Conceder permisos**  
GRANT SELECT, INSERT ON Esquema.Tabla TO AnalistaRol;
- 2. Denegar permisos**  
DENY DELETE ON Esquema.Tabla TO AnalistaRol;
- 3. Revocar permisos**  
REVOKE INSERT ON Esquema.Tabla FROM AnalistaRol;
- 4. Transferir permisos entre roles**  
GRANT EXECUTE ON Procedimiento TO OperadorRol;
- 5. Control total de un objeto**  
GRANT CONTROL ON Esquema.Vista TO AdminRol;

## EJEMPLO DE POLÍTICA RBAC EN UNA ORGANIZACIÓN

| ROL           | DESCRIPCIÓN                            | PERMISOS ASIGNADOS                  | USUARIOS EJEMPLO     |
|---------------|----------------------------------------|-------------------------------------|----------------------|
| Analista      | Puede leer e insertar datos de ventas. | SELECT, INSERT en tablas de ventas  | analista1, analista2 |
| Operador      | Puede actualizar datos operativos.     | SELECT, UPDATE en tablas operativas | operador1, operador2 |
| Supervisor    | Controla y revisa la base de datos.    | CONTROL en bases de datos asignadas | dba_sp, dba_bd       |
| Administrador | Acceso total para gestión y seguridad. | SELECT en todas las tablas          | admin1, admin2       |

La automatización con SQL Server Agent es esencial para mantener la **integridad**, **seguridad** y **eficiencia** de los sistemas de información en entornos empresariales y académicos.

Kevin Alfredo Macha Bruno